

Let the Domain Drive the Design



Object-oriented design, when driven by the domain, decomposes huge monolith systems into a set of agile microservices. Domain-driven design has changed the narration of the object-oriented approach upside down. It has brought the domain objects onto the centrestage of design thinking, and relegated the technology to the peripheral infrastructure. In this fourth part of the 'The Design Odyssey' series, let us examine this approach to software design in some detail.

The Socratic method of learning involves continual probing and cooperative dialogue that is laced with argument. We go by that method to understand the concepts and patterns of domain-driven design. Let's go!

Domain-driven design: Concepts and patterns

What is domain-driven design?

Domain-driven design, or DDD in short, is an approach to designing software systems by applying object-oriented principles in their true spirit.

It implies that DDD is not an alternative to object-oriented design; instead, it only complements it. However, the basic principles like abstraction, encapsulation, inheritance, and polymorphism along with the celebrated SOLID principles already offer a blueprint for object-oriented design. So what else is offered by DDD?

It is true that there are several principles and patterns already in vogue. However, in practice, people often flout most of these principles and patterns, unintentionally. Let's take an example. The principle of encapsulation clearly says that the *state properties* of an object and the *behavioural methods* on those properties must be grouped together within the object itself in such a way that no outsider can directly access the *state* of the object. However, most of the objects we design either contain only state properties or only behavioural methods. The DDD attempts to correct it.

We often design entities and service objects with different responsibilities. An entity consists of just properties that are mapped to the fields of database records. And a service object holds the business logic to be applied to the